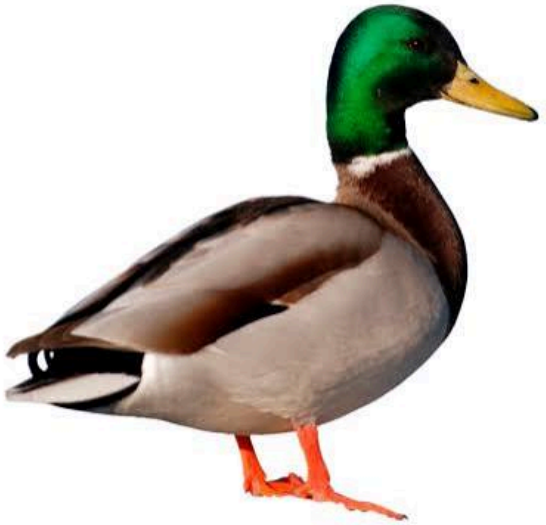
Ciência de Dados Aplicada ao Direito I

Aula 13

Uma resposta para cada caso



Prof. Bruno da Cunha de Oliveira - 2026/2

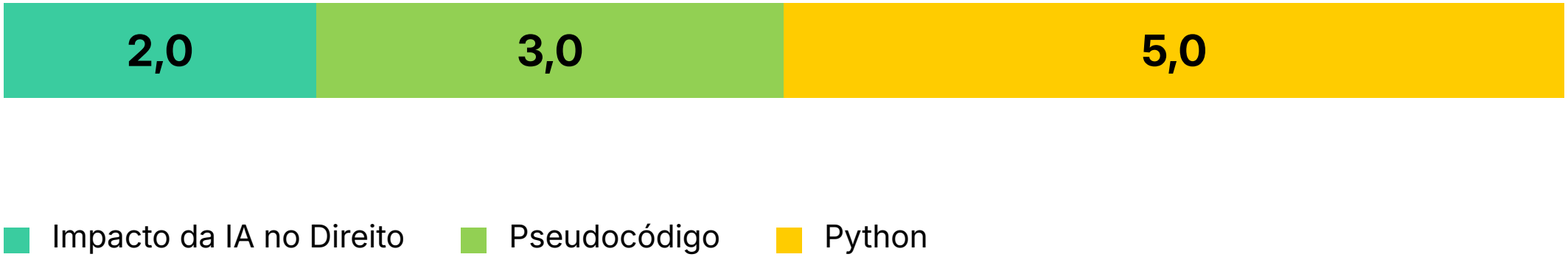
Entrega de hoje: nome, produto e logo

Cada grupo entrega hoje, pelo formulário no Blackboard, o nome da empresa, o nome do produto e a logo.

Cada dia corrido de atraso custa 0,1 na nota da APS, e não há teto.

A logo que entrar hoje é a que vai para a tier list da banca.

Como a prova distribui os 10 pontos



Vou disponibilizar um simulado no Blackboard.

Para a parte de Python, que é metade da prova, estudem pelos notebooks de problemas das aulas.

No PrairieLearn: Desafio II

Insper

Guardar vários registros

Insper

Guardar dois consumidores

```
consumidor = "Ana Lima"
consumidor = "Caio Prado"
print(consumidor)
```

Um núcleo de defesa do consumidor recebeu os pedidos de desistência de Ana e Caio, que compraram por telefone. Este programa guarda os dois nomes?

Guardar dois consumidores

RESPOSTA

```
consumidores = ["Ana Lima", "Caio Prado"]
print(consumidores)
```

```
['Ana Lima', 'Caio Prado']
```

A segunda atribuição substituía Ana por Caio. Uma lista reúne os nomes: colchetes delimitam a coleção e vírgulas separam os itens.

Acrescentar uma consumidora

Chegou o pedido de Bia. Precisamos acrescentar seu nome sem perder os anteriores.

```
consumidores.append("Bia Nunes")
print(consumidores)
```

```
['Ana Lima', 'Caio Prado', 'Bia Nunes']
```

`.append()` é um método da lista, chamado com ponto e parênteses. Acrescenta um item ao fim e altera a própria lista.

Consultar uma posição

A equipe precisa do nome de uma pessoa só, não da lista inteira. Os colchetes depois do nome da lista pedem uma posição.

```
consumidores = ["Ana Lima", "Caio Prado", "Bia Nunes"]
print(consumidores[1])
```

Que nome será mostrado?

Consultar uma posição

RESPOSTA

```
Caio Prado
```

A contagem de posições começa em zero. Ana está na posição 0, e o segundo item está na posição 1.

```
print(consumidores[0])
```

```
Ana Lima
```

Pedir a posição 3

```
consumidores = ["Ana Lima", "Caio Prado", "Bia Nunes"]
print(consumidores[3])
```

A lista tem três nomes. O que a posição 3 devolve?

IndexError: quantidade não é posição

RESPOSTA

```
IndexError: list index out of range
```

Há três itens, nas posições 0, 1 e 2. A posição 3 não existe. Em uma lista não vazia, a última posição é `len(consumidores) - 1`.

Conferir quantidade e presença

Chegou outro pedido em nome de Caio. Antes do `.append()`, a equipe confere se ele já está na lista, para não duplicar.

```
consumidores = ["Ana Lima", "Caio Prado", "Bia Nunes"]
print(len(consumidores))
print("Caio Prado" in consumidores)
print("Davi Rocha" in consumidores)
```

```
3
True
False
```

`len()` conta os itens. `in` testa se um valor está na lista; `not in` testa sua ausência.

Nome e prazo na mesma lista

A equipe quer guardar também se cada pedido chegou dentro dos sete dias. Uma primeira tentativa põe tudo em sequência.

```
pedidos = ["Ana Lima", True, "Caio Prado", False, "Bia Nunes", True]
```

Em que posição está o prazo de Bia? E se cada pedido ganhar também o número dos autos?

Nome e prazo na mesma lista

RESPOSTA

```
print(pedidos[5])
```

```
True
```

Para achar o prazo de Bia foi preciso contar os campos de Ana e de Caio. Com o número dos autos, todas as posições mudariam. Nada na lista diz onde um pedido termina e o outro começa.

Uma lista para cada pedido

Cada pedido vira um registro próprio: uma lista com os autos, o nome e o prazo, guardada dentro da lista de pedidos.

```
pedidos = [
    ["0000123-27.2024.8.99.0001", "Ana Lima", True],
    ["0000456-76.2024.8.99.0001", "Caio Prado", False],
]
pedidos.append(["0000789-25.2024.8.99.0002", "Bia Nunes", True])
print(pedidos[1])
print(pedidos[1][1])
```

```
['0000456-76.2024.8.99.0001', 'Caio Prado', False]
Caio Prado
```

Consultar o registro de prazo

```
pedidos = [
    ["0000123-27.2024.8.99.0001", "Ana Lima", True],
    ["0000456-76.2024.8.99.0001", "Caio Prado", False],
    ["0000789-25.2024.8.99.0002", "Bia Nunes", True],
]
```

Como consultar apenas se o pedido de Bia chegou no prazo?

Consultar o registro de prazo

RESPOSTA

```
print(pedidos[2][2])
```

```
True
```

O primeiro colchete encontra o terceiro pedido. Ele é uma lista; o segundo colchete consulta o registro de prazo dentro dela.

No PrairieLearn: exercício de listas

Insper

Produzir um aviso por pedido

Insper

Uma linha para cada nome?

```
consumidores = ["Ana Lima", "Caio Prado", "Bia Nunes"]
print(consumidores[0])
print(consumidores[1])
```

Falta Bia. Como mostrar todos os nomes sem acrescentar uma linha ao programa a cada consumidor?

Uma linha para cada nome?

RESPOSTA

```
for consumidor in consumidores:
    print(consumidor)
```

```
Ana Lima
Caio Prado
Bia Nunes
```

`for` percorre a lista. A cada volta, `consumidor` recebe o próximo item, e o bloco recuado executa com esse valor.

Consultar os dados de cada pedido

Para decidir sobre os pedidos, só o nome não basta. Vamos percorrer os registros completos.

```
pedidos = [["Ana Lima", True], ["Caio Prado", False]]
for pedido in pedidos:
    print(pedido[0], pedido[1])
```

```
Ana Lima True
Caio Prado False
```

A variável de iteração recebe uma lista inteira. Dentro da volta, consultamos suas posições.

Quantos avisos serão guardados?

A equipe vai devolver um aviso por pedido a quem atende os consumidores. O que o `print()` mostra não fica guardado; os avisos vão para uma lista.

```
pedidos = [["Ana Lima", True], ["Caio Prado", False]]
avisos = []
for pedido in pedidos:
    if pedido[1]:
        aviso = pedido[0] + ": pode desistir"
        avisos.append(aviso)
    else:
        aviso = pedido[0] + ": prazo encerrado"
print(len(avisos))
```

São dois pedidos. Que número será mostrado?

Quantos avisos serão guardados?

RESPOSTA

```
1
```

O aviso de Caio foi montado e nunca entrou na lista: o `.append()` está dentro do ramo do `if`, e só executa quando o prazo está aberto. O programa roda sem erro e entrega um aviso a menos.

Guardar o aviso de cada pedido

```
pedidos = [["Ana Lima", True], ["Caio Prado", False]]
avisos = []
for pedido in pedidos:
    nome = pedido[0]
    no_prazo = pedido[1]
    if no_prazo:
        aviso = nome + ": pode desistir"
    else:
        aviso = nome + ": prazo encerrado"
    avisos.append(aviso)
print(avisos)
```

A lista vazia nasce antes do `for`. O `.append()` fica dentro do `for` e fora dos dois ramos: o `if` escolhe qual aviso, e todo pedido gera um.

Guardar o aviso de cada pedido

Animação no navegador: `animacao-loop.html`, cenário `listas`.

No PrairieLearn: Desafio III

Insper